

To boldly suggest an overall plan for C++20

Abstract

Various people have lamented our lack of direction, and that we don't have a plan for the next standard (or beyond). Since I haven't heard anyone promising to propose such a plan, here goes. In a nutshell, the plan is thus: for C++20, let's work towards having the following things in that standard:

- Modules
- Concepts
- Ranges
- Networking

All good? Agreed? Right, carry on. :) In case you want some elaboration, read on.

Contents

- [Why these things, why not X, Y, and Z?](#)
- [Why Modules?](#)
- [Why Concepts?](#)
- [Why Ranges?](#)
- [Why Networking?](#)
- [Other things](#)

Why these things, why not X, Y, and Z?

So, the first question you all ask, why Modules, Concepts, Ranges and Networking? Why not Contracts? Why not Coroutines? Why not <insert your favorite feature here>?

The facilities I propose are not exhaustive, and do not prevent other things from being included. What I'm suggesting is a list of things that we should focus on having. As has been the case for quite some time, we have a train model, so if something's ready to go and has consensus to be included, by all means let's do so. But let's do our best not to fail (for some cases again) to include something of significant importance.

All the items I mention are reasonably mature, and for the parts where they aren't, we should have sufficient time between now and shipping the C++20 CD to iron out any remaining wrinkles.

Why Modules?

It seems fairly clear that multiple large-scale software houses want Modules. The big benefit is faster builds, but in addition to that, we get various other benefits (better tooling support etc.). Everybody wants faster builds, everybody wants to get something better than textual inclusion. Pending some discussions about certain semantics and migration facilities and whatnot, overall, Modules should be something that we should strive to ship; the improvements they bring are too big to be postponed any further.

I would hope that we can discuss the current points of disagreement in Kona.

Why Concepts?

It would be a surprise to me, actually, if the rationale for Concepts needs further explanation. From a user perspective and a library writer perspective, Concepts provide

- a far superior way to constrain generic interfaces, compared to the hacks we employ today.
- a way to constrain generic interfaces that actually works, whereas the hacks don't really work.

The lack of a way to sanely constrain non-templates (and templates, too), and a lack of a way to have the constraints evaluated at call (overload resolution) time, and the lack of being able to write constraints separate from template parameters, function parameters and return types is painful. Many more users than we expect want to write constrained overload sets, sometimes mutually-exclusive, sometimes not, and telling them how to do so with current work-arounds is incredibly time-consuming.

Concepts are a massive, huge, astonishing improvement to the status quo. There are remaining discussions about whether variable-like and function-like concepts should remain separate, and whether fully-abbreviated syntaxes should be allowed. I would hope that we can discuss those in Kona, we have papers for most of it.

At the risk of sounding like a broken record, I will repeat a particular point: we need to throw definition checking under the bus. If we insist on having it, we will probably get nothing in C++20, and chances are we'll get nothing in C++23. Such a trade-off is unacceptable. The benefits of Concepts as proposed far outweigh the benefits of definition checking, and most users wouldn't care less about not having definition checking.

Why Ranges?

Two reasons:

- Having to always use an iterator pair everywhere is horrible.
- We all want the views and actions - we should get the basic stuff in sooner rather than later so what we can move on to the next step, which will bring the really massive improvement.

Why Networking?

Again, two reasons:

- We have so many networked C++ applications, and tons and tons more to come, that it's time to provide networking support "out of the box".
- Having the (upcoming) Networking TS facilities standardized should open the doors for further development.

Other things

As many will notice, I'm not proposing Contracts, Coroutines, Pattern Matching, Executors and you-name-it to be part of the "master plan". Of those examples, we have certain kinds of Coroutines in a fairly mature shape. The rest are not so mature. I'm not saying we couldn't have such stuff, and I'm not saying we shouldn't have the Coroutines slated for the upcoming TS, but said Coroutines do not cover the vast majority of cases where I want to use Coroutines, and will not do so. They are a high-efficiency facility for a narrow set of uses I need. And most importantly, I don't see them as major as the facilities I propose to be in the "master plan".